

Relatório Final

Desafio de Dados — LH Nautical

José de Oliveira Neto

Agosto de 2026

github.com/JoseNeto1981/lh-nautical-challenge

Sumário Executivo

Este relatório consolida as frentes de trabalho do desafio técnico da LH Nautical, cobrindo o ciclo completo de dados — da ingestão bruta de 24 arquivos CSV à geração de insights preditivos e um sistema de recomendação. O objetivo foi transformar uma base de dados desorganizada em uma fonte confiável de decisão para a diretoria, com clareza técnica e impacto de negócio mensurável em cada etapa, seguindo as premissas definidas pelo Tech Lead Gabriel Santos.

O que foi entregue

- Tratamento de Dados (EDA, Schema e Carregamento): diagnóstico de qualidade dos dados brutos, geração automatizada de schema PostgreSQL a partir dos CSVs, e carga completa de 251.864 linhas em um banco relacional real, rodando em container Docker.
- Análise de Clientes, Análise de Vendas, Previsão de Demanda e Sistemas de Recomendação: identificação dos clientes mais fiéis e o que eles consomem, correção de um erro de cálculo que distorcia a análise de vendas por dia da semana, um modelo preditivo de demanda com métrica de erro, e um motor de recomendação de produtos baseado em comportamento de compra.

Principais achados para a diretoria

#	Insights	Impacto
1	A base de dados brutos tem boa integridade (sem nulos, sem negativos), mas contém uma data futura suspeita e ao menos um caso de produto duplicado no cadastro.	Confiabilidade dos dados
2	Entre os 10 clientes mais fiéis (maior ticket médio, compram em 14+ categorias), Hélices é a categoria mais vendida.	Estratégia comercial
3	Corrigido o erro do estagiário: Quinta-feira é o pior dia de vendas, não Domingo. O Domingo fica em segundo pior lugar, não em primeiro melhor.	Decisão operacional
4	O baseline de previsão (média móvel) subestimou sistematicamente as vendas do 1º trimestre de 2026 (149 previstas vs. 207 reais).	Risco de ruptura de estoque
5	A recomendação por similaridade aponta outro motor de popa, não uma defesa, como produto mais associado ao Motor de Popa 1949.	Estratégia de cross-sell

Tabela 1 — Síntese dos principais achados do desafio

O relatório está organizado por frente de trabalho, cada uma com metodologia, decisões técnicas, resultados e — quando aplicável — visualizações de apoio.

Como o Projeto Foi Construído

O desafio foi resolvido como um pipeline de dados de ponta a ponta, passando por cinco etapas principais: leitura dos CSVs brutos, exploração inicial, modelagem de schema, carga em um banco relacional real (PostgreSQL, rodando em Docker), e finalmente as quatro análises de negócio solicitadas.

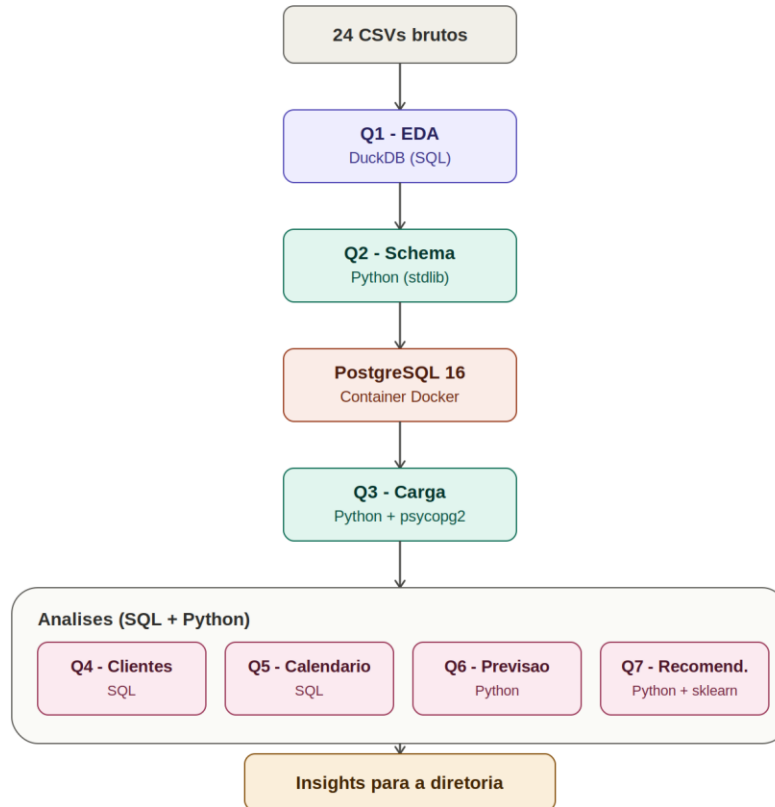


Figura 1 — Fluxograma geral do pipeline de dados

O Tratamento de Dados (EDA, Schema e Carregamento) seguiu um caminho técnico específico: a EDA inicial foi feita com DuckDB direto sobre os arquivos CSV (sem precisar de banco ainda), a geração do schema usou exclusivamente bibliotecas nativas do Python, e a carga final usou o comando COPY nativo do PostgreSQL via psycopg2, orquestrado por um container Docker.

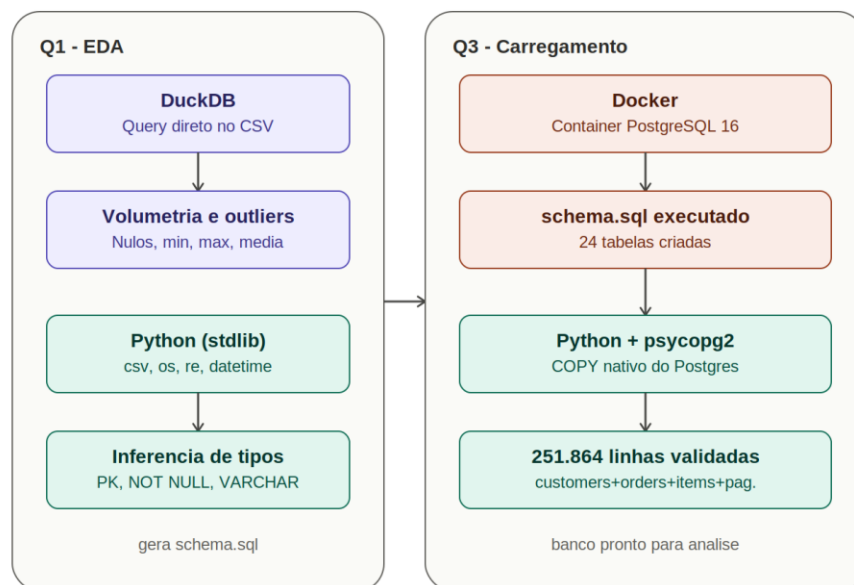


Figura 2 — Detalhamento técnico da frente de Tratamento de Dados

As frentes de Análise de Clientes, Análise de Vendas, Previsão de Demanda e Sistemas de Recomendação partiram todas do mesmo banco já carregado, cada uma aplicando uma técnica diferente — desde SQL analítico com CTEs até um modelo preditivo simples e um sistema de recomendação com similaridade de cosseno.

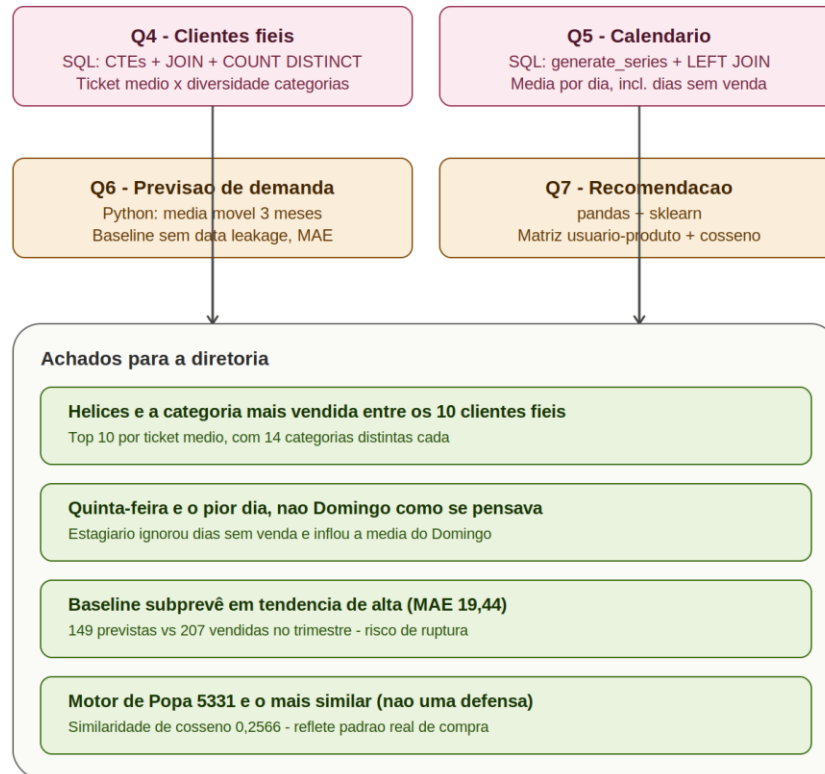


Figura 3 — Detalhamento técnico das frentes de análise de negócio

Frente 1 — Análise Exploratória de Dados (EDA)

Referência: Questão 1 do desafio

Pergunta de negócio

O Sr. Almir quer uma resposta simples: "Posso confiar nesses dados para tomar decisões?"

O que foi feito

Uma análise exploratória inicial na tabela orders, usando DuckDB para consultar o CSV diretamente (sem necessidade de carregar em banco ainda), cobrindo volume, distribuição de valores e qualidade dos dados.

Resultados

Métrica	Valor
Total de linhas	48.998
Total de colunas	13
Período (created_at)	2020-01-01 a 2026-12-31
Valor mínimo (total)	R\$ 32,62
Valor máximo (total)	R\$ 127.262,02
Valor médio (total)	R\$ 28.704,99
Nulos em total / created_at	0
Negativos em total	0

Tabela 2 — Visão geral da tabela orders

Diagnóstico de confiabilidade

A base tem boa integridade estrutural nas colunas auditadas, mas dois pontos exigem atenção antes de decisões críticas:

- A distância entre a média (R\$ 28.704,99) e o máximo (R\$ 127.262,02) sugere distribuição assimétrica — poucos pedidos de alto valor distorcem a leitura de "pedido típico". Recomenda-se uso de mediana/percentis para análises futuras baseadas em valor médio.
- A data máxima (31/12/2026) está no futuro em relação à data de hoje — um forte indício de dado sintético (esperado em um desafio), mas que exigiria investigação se fosse produção real.

Resposta direta ao Sr. Almir: os dados são utilizáveis para uma análise exploratória inicial, mas não é recomendado tomar decisões estratégicas com base neles sem antes tratar os outliers e esclarecer a questão da data futura.

Frente 2 — Tratamento de Dados: Modelagem de Schema

Referência: Questão 2 do desafio

Pergunta de negócio

Como o ERP não permite conexão direta ao banco, os dados só existem como CSVs. Era preciso definir a estrutura (schema) de um banco PostgreSQL para receber esses dados.

O que foi feito

Um script em Python puro (apenas bibliotecas nativas: csv, os, re, glob, datetime) lê todos os CSVs de um diretório, infere o tipo de dado PostgreSQL mais adequado para cada coluna, e gera um único arquivo schema.sql com os CREATE TABLE correspondentes.

Decisões técnicas de destaque

- Identificadores como texto, não número: colunas como cpf, phone, tax_id e barcode_ean foram tratadas como VARCHAR, não BIGINT — evitando perda de zeros à esquerda e refletindo que esses campos nunca são usados em cálculo.
- Boolean priorizado sobre inteiro: a ordem de verificação de tipo garante que colunas binárias (0/1, true/false) sejam corretamente identificadas como BOOLEAN, não INTEGER.
- Chave primária e obrigatoriedade inferidas automaticamente: a coluna id é marcada como PRIMARY KEY quando única e sem nulos; colunas sem nenhum valor vazio na amostra recebem NOT NULL.

Validação

O schema.sql gerado foi executado de fato contra um PostgreSQL 16 real (container Docker), não apenas gerado teoricamente. Resultado: 24 tabelas criadas com sucesso, sem nenhum erro de sintaxe.

Frente 2 — Tratamento de Dados: Carregamento

Referência: Questão 3 do desafio

Pergunta de negócio

Com o schema criado, era necessário carregar os dados reais dos CSVs no banco, sem nenhum tratamento (sem remoção de nulos, sem correção de caracteres) — carga bruta, fiel à origem.

O que foi feito

Um script Python usando psycopg2 e o comando COPY nativo do PostgreSQL — a forma mais rápida e menos invasiva de carregar CSVs em massa, já que o próprio banco interpreta os tipos a partir do texto, sem nenhuma lógica de conversão passando pela mão do Python.

Resultado

Todos os 24 arquivos CSV foram carregados com sucesso, totalizando volumes que variam de 6 linhas (locations) a 147.320 linhas (order_items).

Validação de volumetria

Pergunta: soma total de linhas entre customers, orders, order_items e payments?

Tabela	Linhas
customers	2.000
orders	48.998
order_items	147.320
payments	53.546
Total	251.864

Tabela 3 — Validação de volumetria pós-carga

Esse valor foi confirmado rodando a query diretamente contra o banco carregado, batendo exatamente com a soma esperada — confirmando que a carga não perdeu nem duplicou nenhuma linha.

Frente 3 — Análise de Clientes

Referência: Questão 4 do desafio

Pergunta de negócio

A Diretoria da LH Nautical deseja identificar os clientes fiéis: não quem compra muito uma única vez, mas quem tem gasto médio alto por transação e navega por diversas categorias da loja — para replicar esse comportamento em outros segmentos.

O que foi feito

Uma análise SQL com CTEs (common table expressions) calculando, por cliente: Faturamento Total (soma de total em orders), Frequência (contagem de pedidos), Ticket Médio (Faturamento / Frequência) e Diversidade de Categorias (COUNT DISTINCT category_id, percorrendo orders → order_items → product_variants → products). Os clientes foram filtrados pelo critério de elite (diversidade ≥ 13 categorias) e ranqueados pelo maior Ticket Médio.

Resultado — os 10 clientes fiéis

customer_id	Faturamento (R\$)	Frequência	Ticket Médio (R\$)	Diversidade
22	1.087.838,44	26	41.839,94	14
1477	916.262,58	22	41.648,30	14
929	1.082.775,89	26	41.645,23	14
1116	655.737,20	16	40.983,58	14
1691	815.471,30	20	40.773,57	14
774	726.127,99	18	40.340,44	14
1470	1.040.553,09	26	40.021,27	14
1599	997.616,46	25	39.904,66	14
965	677.297,78	17	39.841,05	14
1722	1.146.455,22	29	39.532,94	14

Tabela 4 — Os 10 clientes fiéis, ordenados por Ticket Médio

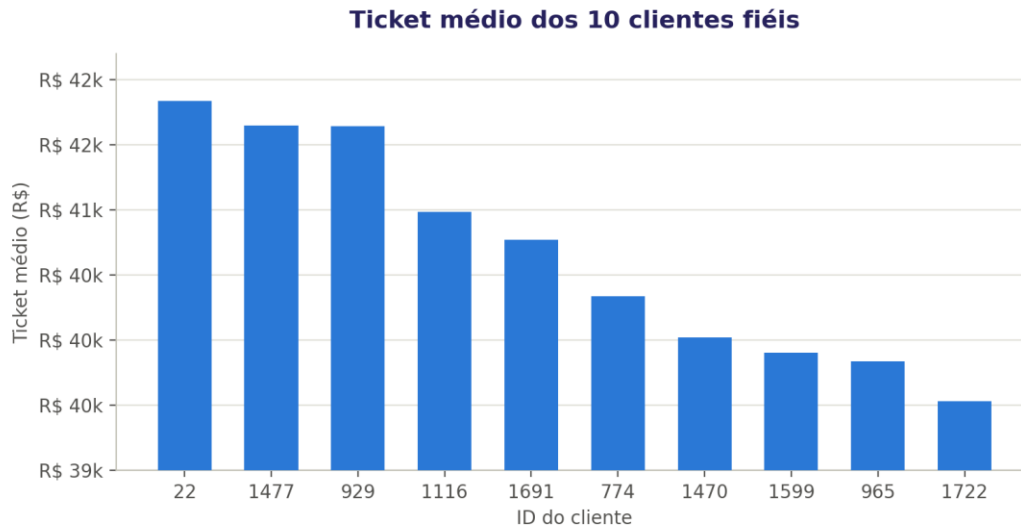


Figura 4 — Ticket médio dos 10 clientes fiéis

O gráfico evidencia a proximidade entre os 10 clientes: a diferença entre o maior e o menor ticket médio do grupo é de apenas R\$ 2.307, um intervalo estreito que reforça a coesão desse segmento de elite.

Observação: todos os 10 clientes fiéis apresentam diversidade exatamente igual a 14 categorias — nenhum com o mínimo de 13, nenhum com 15 ou mais. Isso sugere que 14 pode ser um teto natural de diversidade no catálogo da loja para o perfil de maior ticket médio.

Categoria mais vendida entre os clientes fiéis

Entre os itens comprados por esses 10 clientes, a categoria que concentra a maior quantidade total de itens (SUM(quantity)) é:

Categoria	Total de itens comprados
Hélices	492

Tabela 5 — Categoria mais vendida entre os clientes fiéis

Conclusão: Hélices é a categoria com maior concentração de compra entre o grupo de elite — uma pista concreta de onde focar esforços de retenção e cross-sell direcionados a esse perfil de cliente.

Frente 4 — Análise de Vendas: Dimensão de Calendário

Referência: Questão 5 do desafio

Pergunta de negócio

O Sr. Almir quer saber: "Qual é o dia da semana, nas lojas físicas, temos a pior média de vendas?" para decidir se vale a pena fechar a loja nesses dias.

O erro identificado

Um estagiário calculou a média de vendas por dia da semana agrupando diretamente a tabela orders, e concluiu que Domingo era o melhor dia (média de R\$ 5.000,00). O problema: em muitos domingos a loja abriu mas vendeu zero — e, como esses dias não geram nenhuma linha em orders, eles foram silenciosamente excluídos do cálculo, inflando artificialmente a média.

O que foi feito

Uma dimensão de calendário foi construída via `generate_series()`, cobrindo todos os dias entre a menor e a maior data de venda (2.557 dias). Essa dimensão foi então cruzada com as vendas de lojas físicas (`channel = 'pos'`) via `LEFT JOIN`, substituindo dias sem venda por 0 (`COALESCE`) — garantindo que todo dia do calendário, com ou sem venda, entrasse no cálculo da média.

Resultado corrigido

Dia da semana	Dias no período	Média de vendas/dia (R\$)
Quinta-feira	366	157.154,32
Domingo	365	157.616,13
Segunda-feira	365	158.241,15
Sábado	365	164.858,27
Terça-feira	365	166.118,83
Sexta-feira	365	170.193,68
Quarta-feira	366	173.605,44

Tabela 6 — Média de vendas por dia da semana (ordenado do pior para o melhor)

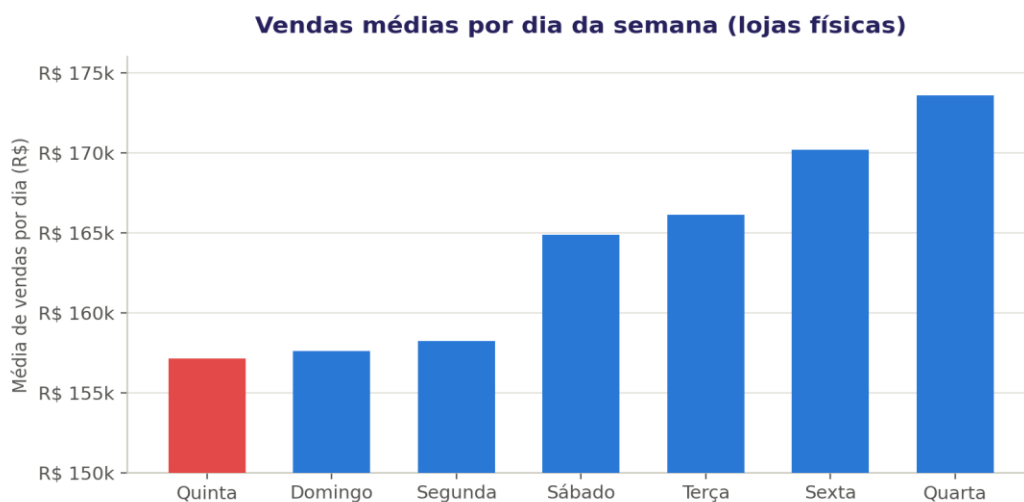


Figura 5 — Vendas médias por dia da semana, lojas físicas

Em destaque, Quinta-feira aparece como o dia de menor média — mas o gráfico também deixa visível que os três primeiros colocados (Quinta, Domingo, Segunda) estão muito próximos entre si, formando um patamar de baixo desempenho, enquanto Quarta-feira se destaca isoladamente como o melhor dia.

Conclusão: o pior dia de vendas nas lojas físicas é Quinta-feira (R\$ 157.154,32 de média), não Domingo. O Domingo, na verdade, é o segundo pior — sua média real (R\$ 157.616,13) é cerca de 31 vezes maior que o número reportado pelo estagiário, que havia excluído incorretamente os domingos sem venda do cálculo.

Frente 5 — Previsão de Demanda

Referência: Questão 6 do desafio

Pergunta de negócio

No último verão, o estoque de "Coletes Salva-Vidas" esgotou em 3 meses, gerando perda de vendas — enquanto "Âncoras" foram compradas em excesso e ficaram paradas no galpão. O Tech Lead Gabriel Santos quer um modelo preditivo que diga quantas unidades serão vendidas no próximo mês, para ajustar compras com fornecedores em vez de decidir por "feeling".

Achado prévio: ambiguidade de cadastro

Antes de construir o modelo, foi identificado que existem dois produtos distintos na base (product_id 74 e 240), com SKUs, preços e histórico de vendas próprios, mas exatamente o mesmo nome: "Bússola de Bordo 702". Ambos têm vendas desde 2020, descartando a hipótese de um ser sucessor do outro — é aparentemente uma falha de cadastro no ERP de origem. Como o enunciado se refere ao produto pelo nome, a decisão foi agregar as vendas de ambos os product_id como uma única série temporal, documentando essa ambiguidade para o time de dados investigar.

O que foi feito

Um script Python (psycopg2 para consultar o banco) construiu a série mensal de vendas do produto, unindo orders, order_items, product_variants e products. O baseline utilizado foi uma média móvel de 3 meses, sempre considerando apenas dados anteriores à data prevista (sem data leakage) — um rolling forecast onde o valor real de cada mês, assim que conhecido, passa a integrar a janela do mês seguinte.

Previsão vs. Realizado — 1º trimestre de 2026

Mês	Previsto	Realizado	Erro Absoluto
2026-01	38,67	79	40,33
2026-02	53,67	68	14,33
2026-03	56,33	60	3,67
Soma do trimestre	149	207	—

Tabela 7 — Previsão (média móvel 3 meses) vs. vendas reais

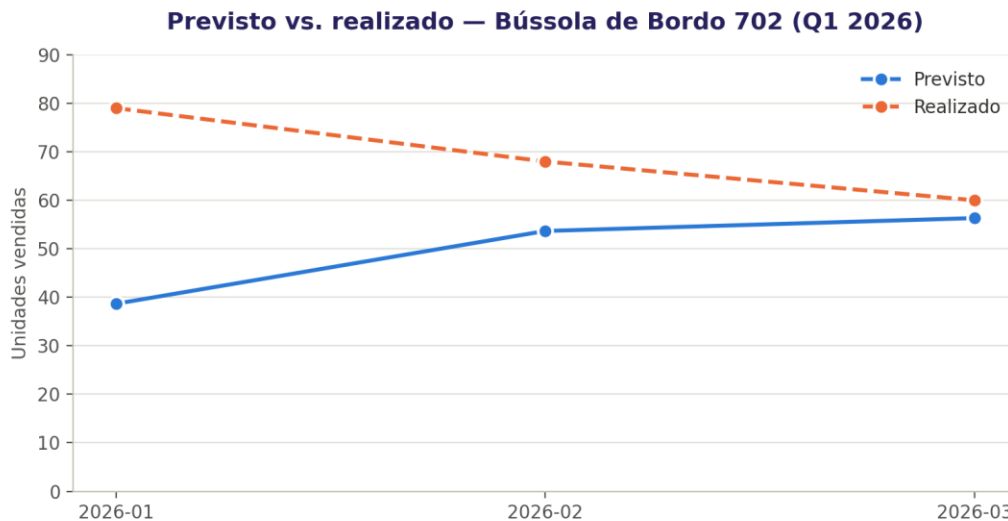


Figura 6 — Previsto vs. realizado, Bússola de Bordo 702 (Q1 2026)

O gráfico torna visível o padrão de subestimação: a linha de previsão (azul) permanece sistematicamente abaixo da linha de vendas reais (laranja) nos três meses, embora a distância entre elas diminua ao longo do trimestre — sinal de que o modelo vai se ajustando, mas sempre com atraso em relação à tendência real.

MAE (Mean Absolute Error): 19,44 unidades — em média, cada previsão mensal ficou a cerca de 19-20 unidades de distância do valor real, entre 25% e 32% do valor real de cada mês.

O baseline é adequado para esse produto?

Não é totalmente adequado. O modelo subestimou sistematicamente em todos os 3 meses (sempre previsto abaixo do real), somando 58 unidades de diferença no trimestre — quase 28% abaixo do realizado. Isso indica uma tendência de crescimento na demanda que a média móvel, por natureza, não conseguiu antecipar: ela sempre "olha para trás".

Limitação do método

A média móvel de 3 meses não captura sazonalidade nem tendência. O próprio cenário do desafio (ruptura de estoque no verão) já é um sinal de que a demanda de produtos náuticos tem picos sazonais previsíveis — algo que esse baseline simples não incorpora.

Frente 6 — Sistemas de Recomendação

Referência: Questão 7 do desafio

Pergunta de negócio

A Marina quer implementar uma vitrine de "Quem comprou isso, também levou..." no site, identificando qual produto recomendar junto ao "Motor de Popa 1949", com base na similaridade de comportamento de compra dos clientes.

O que foi feito

Uma matriz de interação Usuário × Produto foi construída com pandas (1 se o cliente comprou o produto ao menos uma vez, 0 caso contrário — ignorando quantidade). A partir dela, a similaridade de cosseno entre produtos foi calculada com scikit-learn (cosine_similarity), comparando os vetores de clientes que compraram cada par de produtos.

Resultado

Matriz de 2.000 clientes × 500 produtos, construída a partir de 135.508 pares distintos (cliente, produto).

Posição	Produto	Similaridade (cosseno)
1	Motor de Popa 5331	0,2566
2	Cabo Náutico 2105	0,2562
3	Vela Mestra 1913	0,2558
4	Cabo Náutico 9048	0,2393
5	GPS Plotter 6249	0,2377

Tabela 8 — Top 5 produtos mais similares ao Motor de Popa 1949

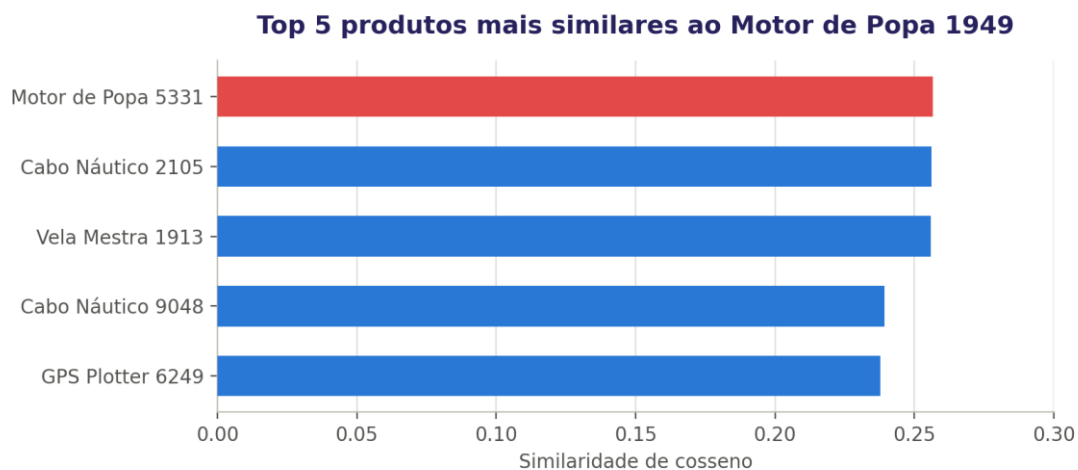


Figura 7 — Ranking de similaridade de cosseno

O gráfico mostra que os cinco produtos do ranking têm similaridade muito próxima entre si (0,2377 a 0,2566) — reforçando que, nesse catálogo, nenhuma associação de produtos é fortemente dominante, e a diferença entre a 1ª e a 5ª posição é pequena o suficiente para que outras regras de negócio possam complementar a recomendação.

Produto com maior similaridade: Motor de Popa 5331, com similaridade de cosseno de 0,2566.

O que a similaridade de cosseno significa aqui

Ela mede o quão parecido é o padrão de clientes que compraram dois produtos — tratando cada produto como um vetor onde cada posição representa um cliente (1 se comprou, 0 se não). Quanto mais próximos de 1, mais os mesmos clientes tendem a comprar ambos os produtos.

Limitação do método

O modelo captura correlação estatística de compra, mas não tem nenhum mecanismo para incorporar conhecimento de domínio — ele não "sabe" que um motor e uma defesa são fisicamente complementares (um protege o barco onde o outro é instalado); só enxerga o padrão de compra histórico. É por isso que o resultado aponta outro motor de popa, e não uma defesa como a Marina hipotetizou: é uma limitação estrutural de qualquer recomendação baseada apenas em co-ocorrência de compra, sem contexto de produto.

Conclusão e Recomendações

O desafio evidenciou, de ponta a ponta, como decisões aparentemente simples de análise de dados podem levar a conclusões erradas quando o método não é rigoroso — e como um pipeline bem estruturado corrige isso. Três exemplos concretos surgiram ao longo do trabalho:

- O erro do estagiário na análise de vendas mostra como ignorar "ausência de dado" (dias sem venda) pode inflar artificialmente uma métrica de negócio, levando a uma decisão operacional equivocada (fechar a loja no dia errado).
- A investigação de cadastro na frente de Previsão de Demanda (dois produtos com o mesmo nome) reforça que qualidade de dado não é um detalhe técnico — é pré-requisito para qualquer análise ou modelo confiável.
- O modelo de previsão de demanda e o sistema de recomendação são, ambos, ferramentas úteis mas limitadas: eles substituem o "feeling" por uma metodologia auditável, mas exigem acompanhamento humano — o baseline de média móvel tende a errar em períodos de mudança de tendência, e a recomendação por similaridade não capta complementaridade física de produtos.

Recomendações para os próximos passos

- Investigar e corrigir a duplicidade de cadastro identificada (produto "Bússola de Bordo 702" com dois IDs distintos) e auditar se há outros casos semelhantes na base.
- Evoluir o modelo de previsão de demanda para incorporar sazonalidade (ex: médias móveis ponderadas por período do ano, ou modelos que decomponham tendência e sazonalidade), reduzindo o risco de ruptura de estoque em períodos de alta.
- Complementar o sistema de recomendação por similaridade com regras de negócio manuais para categorias de complementaridade física conhecida (ex: motor + defesa, lancha + colete salva-vidas), já que o modelo estatístico sozinho não capta esse tipo de relação.
- Esclarecer a data futura encontrada na EDA inicial (created_at até 31/12/2026) antes de qualquer análise temporal ser usada em decisões de produção real.